

Lottery Search Benchmark

ค้นสลาก 6 หลักจำนวน **10 ล้านใบ** ด้วย pattern ที่มี wildcard เช่น * * * * 2 3 แล้วแจกให้ผู้ใช้พร้อมกันหลายคนโดยห้ามซ้ำ เอกสารนี้เทียบ 4 วิธีทำ index ด้วยตัวเลขที่วัดจริงบน Kubernetes ที่บังคับ resource limit ทุกตัวเลขมีไฟล์หลักฐานกำกับ

วันที่วัด	ข้อมูล	API POD	POSTGRES POD
2026-09-04	10,000,000 ใบ	500m · 128Mi	2000m · 1Gi
เครื่องมือ	SCENARIO		
k6 v1.2.3	7 ชุด		

สมมติฐาน 3 ข้อผ่านหมด ด้วย Strategy C

H1 · LATENCY ผ่าน p95 56 ms เป้าคือ p95 ต่ำกว่า 100 ms ทุกคลาส pattern ที่ 100 VUs ทำได้ 5,346 req/s error 0	H2 · NO DUPLICATE ผ่าน 0 ใบซ้ำ จอง 351,481 ใบ ภายใต้ 200 VUs ที่ยิง pattern เดียวกัน ไม่มีตัวใบไหนไปถึงสองคน	H3 · FASTER THAN LIKE ผ่าน ~15,000× pattern แบบระบุครบ 6 หลัก จาก 955 ms เหลือ 0.06 ms และ index เล็กกว่า Strategy A ครึ่งหนึ่ง
STABILITY ผ่าน 27 / 128 MiB soak 10 นาที 744k requests ไม่ restart ไม่ OOMKilled memory หนึ่งที่ราวหนึ่งในห้าของ limit		

สิ่งที่แนะนำให้ใช้ คือ PostgreSQL ตัวเดียว กับ **Strategy C** สำหรับการค้นหา และใช้ **สุ่มเลขจริงใน pattern แล้ว point lookup** คู่กับ `FOR UPDATE SKIP LOCKED` สำหรับการจอง ไม่ต้องมี Redis ไม่ต้องมี distributed lock

01 โจทย์และกฎแฉสำคัญ

พื้นที่ค้นหาจริงคือ 1 ล้าน ไม่ใช่ 10 ล้าน

สลาก 6 หลักมีค่าที่เป็นไปได้แค่ $10^6 = 1,000,000$ เลข เมื่อมีสลาก 10 ล้านใบ แปลว่า เลขหนึ่งเลขมีสลากเฉลี่ย 10 ใบ ดังนั้นการค้นหา pattern จึงไม่จำเป็นต้องไล่ดูสลากทั้ง 10 ล้านใบ แค่หาว่า *เลขไหนบ้างที่เข้า pattern* จากตารางเล็ก 1 ล้านแถวก่อน แล้วค่อย join เข้าตารางสลาก นี่คือการคิดเบื้องหลัง Strategy B และ C

อีกเรื่องที่ต้องเข้าใจก่อนอ่านตัวเลข คือขนาดผลลัพธ์ต่างกันเป็นล้านเท่าตามจำนวน wildcard เราจึงแบ่ง pattern เป็น 4 คลาสแล้ววัดแยกกัน

1 2 3 4 5 6

w0 — ระบุครบ ไม่มี wildcard เข้าเงื่อนไข 1 เลข รว 10 ใบ นี่คือการที่แย่ที่สุดสำหรับ index ที่ไม่ดี

1 2 3 * * *

w3 — ล็อกสามหลักหน้า เข้าเงื่อนไข 1,000 เลข รว 10,000 ใบ

* * * * 2 3

w4 — ลงท้ายด้วย 23 เข้าเงื่อนไข 10,000 เลข รว 100,000 ใบ

* * 3 * * *

w5 — ล็อกหลักกลางหลักเดียว เข้าเงื่อนไข 100,000 เลข รว 1,000,000 ใบ

ทำไม B-TREE ธรรมดาไม่พอ

index ปกติบนคอลัมน์ `number` เรียงจากซ้ายไปขวา จึงใช้ได้เฉพาะตอนที่รู้หลักหน้า เช่น

1 2 3 * * *

แต่ถ้า wildcard อยู่หน้าอย่าง * * * * 2 3 index จะช่วยอะไรไม่ได้เลย นี่คือหัวใจของโจทย์

02 4 กลยุทธ์ INDEX

แต่ละแบบทำอะไร และแลกอะไรไป

ทั้ง 4 แบบใช้ schema เดียวกัน ต่างกันแค่ index และวิธีเขียน query สลับด้วย env var `STRATEGY` ได้โดยไม่ต้อง migrate ใหม่ ทำให้เทียบกันบนข้อมูลชุดเดียวกันได้จริง

กลยุทธ์	ทำอย่างไร	ขนาด INDEX	จุดอ่อน
0 Baseline ตัวเปรียบเทียบ	<code>lpad(number::text,6,'0') LIKE '____23'</code>	—	ไม่มี index ไหนใช้ได้ ต้องสแกนทั้งตาราง
A Expression index index ต่อหลัก	สร้าง index แยกที่ละหลักบนตาราง <code>tickets</code> แล้วหวังให้ planner ทำ BitmapAnd	397 MB	หลักเดียวคัดได้แค่ 1 ใน 10 planner จึงมักเลือกใช้ index เดียวแล้ว filter ที่เหลือ
B Dimension table ตาราง 1 ล้านแถว	หาเลขที่เข้า pattern จากตาราง <code>numbers(n, d1..d6)</code> ก่อนแล้ว join เข้า <code>tickets</code>	192 MB	ยังต้อง BitmapAnd หลาย digit index บนตาราง <code>numbers</code>
C B + prefix range แนะนำ	เหมือน B แต่แปลงหลักหน้าที่ล็อกไว้เป็นช่วง <code>n BETWEEN</code> บน primary key และถ้าระบุครบ 6 หลักก็ยิง <code>number = X</code> ตรงๆ	192 MB	ถ้า wildcard อยู่หน้าหมด จะกลับไปพึ่ง digit index เหมือน B

ขนาด index วัดด้วย `pg_relation_size` ที่ 10M rows ส่วน heap ของ `tickets` เองอยู่ที่ 422 MB

ประเด็นสำคัญของ Strategy C คือมันเปลี่ยนงานจาก "ไล่หาทีละหลัก" เป็น "อ่านช่วงเดียวบน primary key" ซึ่ง Postgres ทำได้เร็วมากและเป็น index-only scan ด้วย

```
-- pattern 123*** ภายใต้ Strategy C
number IN (SELECT n FROM numbers WHERE n BETWEEN 123000 AND 123999)

-- pattern 123456 ภายใต้ Strategy C ข้ามตาราง numbers ไปเลย
number = 123456
```

03 ผล EXPLAIN

วัดทีละ query ยังไม่มีโหลด

ก่อนจะยิงโหลด ต้องรู้ก่อนว่า query เดี่ยวๆ ใช้เวลาเท่าไร ตัวเลขนี้มาจาก EXPLAIN (ANALYZE, BUFFERS) บน 10M rows โดยขอผลแค่ LIMIT 20 หน่วยเป็นมิลลิวินาที

PATTERN	WILDCARD	0 BASELINE	A	B	C
123456	0	955	163	5.5	0.06
123***	3	2.6	19.9	3.0	0.12
****23	4	0.28	0.88	3.4	0.13
3*	5	0.05	0.03	1.1	0.20
*****	6	0.03	0.02	0.02	0.02

จุดที่คนอ่านผิดบ่อย

baseline ดู "เร็ว" ในแถวล่างๆ ไม่ใช่เพราะมันดี แต่เพราะมี LIMIT 20 pattern ที่กว้างมากจะเจอ 20 ไบแรกตั้งแต่ต้นตาราง เลยหยุดสแกนเร็ว เสร็จที่วัดฝีมือจริงคือแถวบนสุด 1 2 3 4 5 6 ที่มีของตรงเงื่อนไขแค่ ~10 ไบใน 10 ล้านไบ ทำให้ต้องอ่านทั้งตารางกว่าจะครบ 20

ไต่จาก 10 ถึง 100 VUs ยิง /v1/search

รันสักรอบ ข้อมูลชุดเดียวกัน pod เดียวกัน เปลี่ยนแค่ STRATEGY แต่ละรอบสุ่ม pattern จาก 4 คลาส แล้วแยกสถิติตามคลาส หน่วยเป็นมิลลิวินาที

กลยุทธ์	P95 W0	P95 W3	P95 W4	P95 W5	P95 รวม	สูงสุด	REQ/S	ERROR	H1
C numbers + prefix range	56.3	55.9	55.5	55.6	55.8	142	5,346	0 %	ผ่าน
B numbers table	147.1	144.6	108.4	108.7	142.3	199	926	0 %	ไม่ผ่าน
A expression index ต่อหลัก	6,718	5,162	4,859	4,860	5,972	8,151	22	0 %	ไม่ผ่าน
O baseline LIKE	1,053	1,021	1,023	1,021	1,022	22,550	145	95.7 %	ล้ม

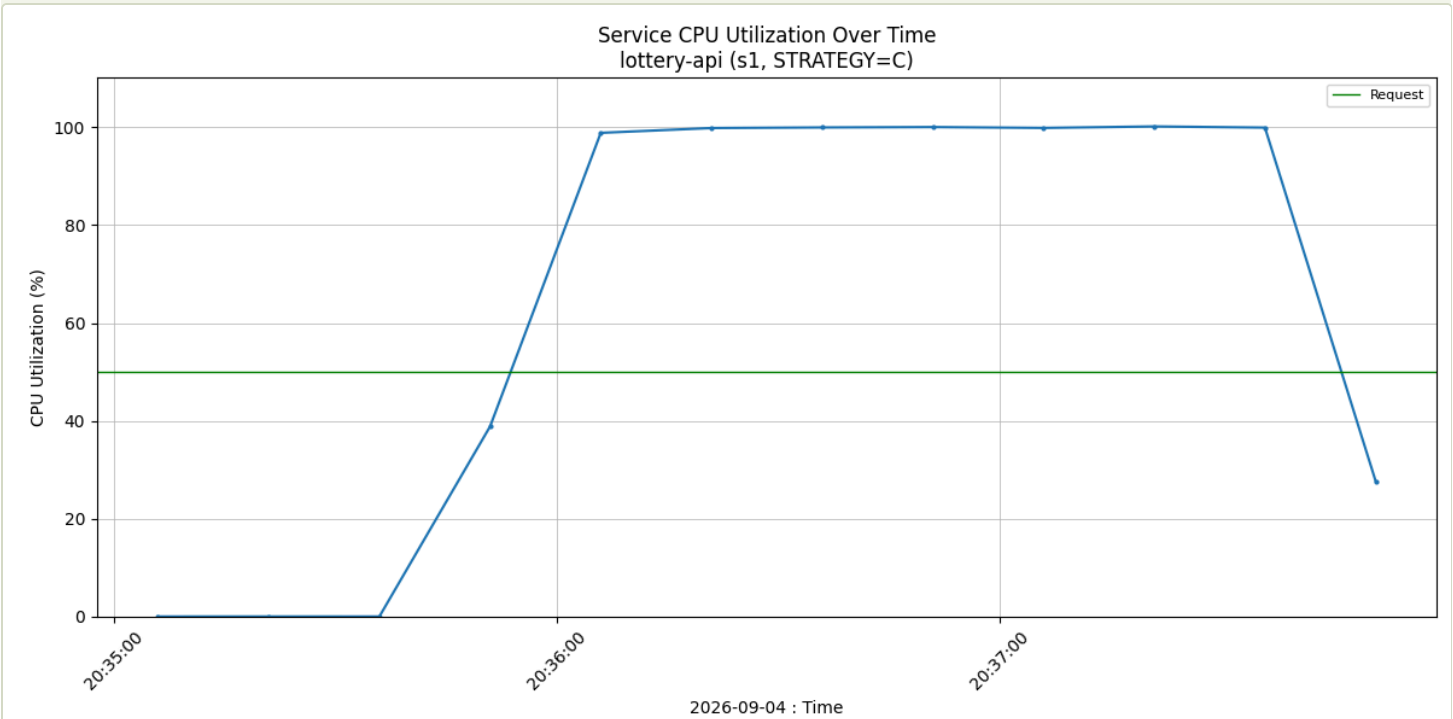
p95 อ่านว่า 95 % ของ request เร็วกว่าค่านี้ ใช้แทนค่าเฉลี่ยเพราะค่าเฉลี่ยซ่อนหางยาวที่ผู้ใช้เจอจริง

คอขวดอยู่ตรงไหน

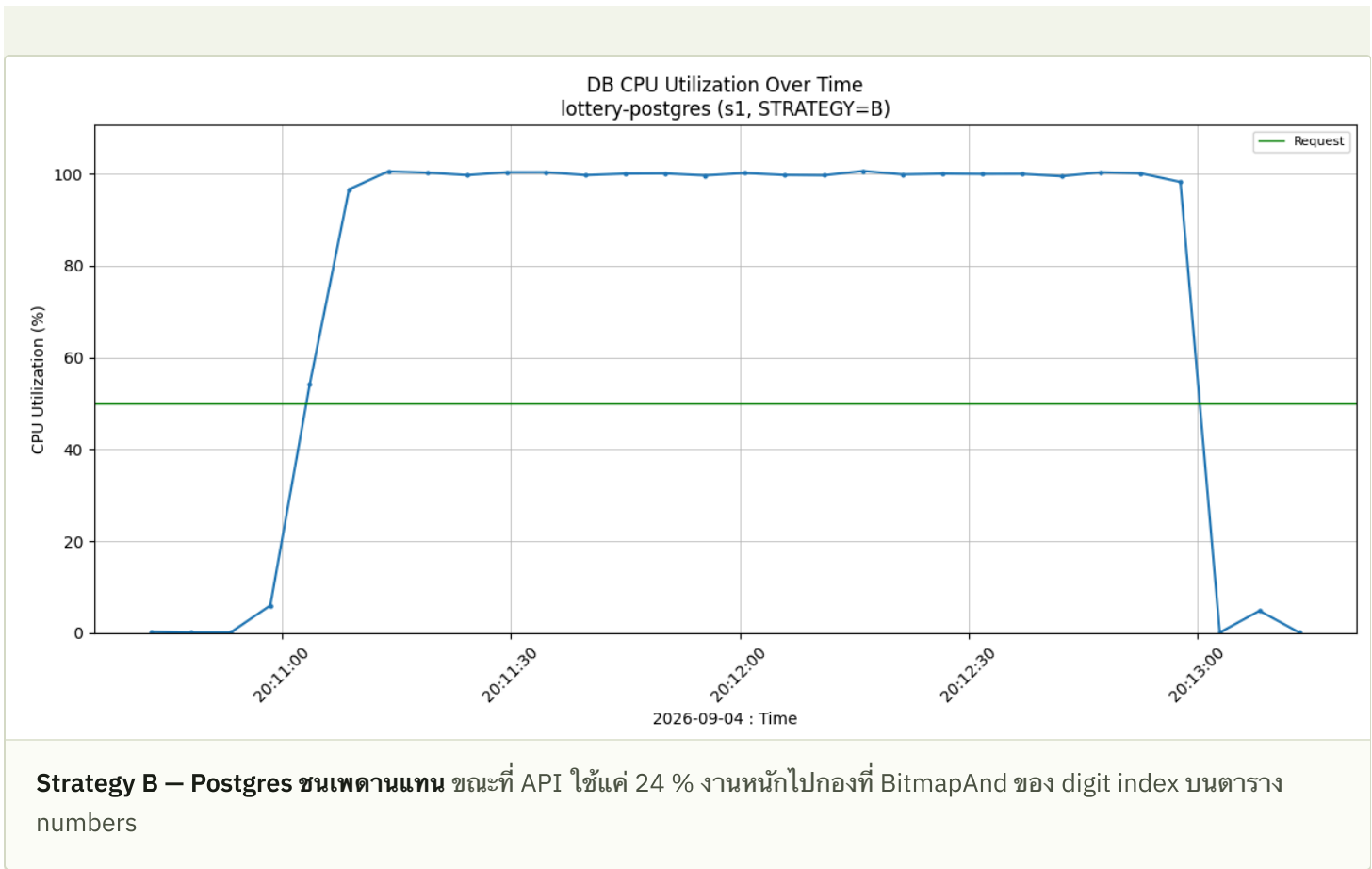
ตารางข้างล่างสำคัญพอๆ กับตาราง latency เพราะมันบอกว่าทรัพยากรตัวไหนหมดก่อน ซึ่งเป็นตัวกำหนดว่าจะขยายระบบต่ออย่างไร

กลยุทธ์	API CPU	API MEM	POSTGRES CPU	POSTGRES WORKING SET	POSTGRES เหลือว่าง
C	100 % ขนเพดาน	19 MiB	88 %	823 MiB	203 MiB
B	24 %	17 MiB	101 % ขนเพดาน	745 MiB	279 MiB
A	1 %	12 MiB	101 % ขนเพดาน	1,016 MiB	8 MiB
O	1 %	9 MiB	102 % ขนเพดาน	456 MiB	568 MiB

C เป็นแบบเดียวที่ย้ายคอขวดออกจาก database ได้ พอ Postgres ไม่ใช่ตัวที่เต็มแล้ว การเพิ่ม replica ของ API จะทำให้ throughput ขึ้นต่อได้ทันที ส่วน B กับ A ต่อให้เพิ่ม API อีกก็ตัวก็ไม่ช่วย เพราะ Postgres เต็มไปแล้ว

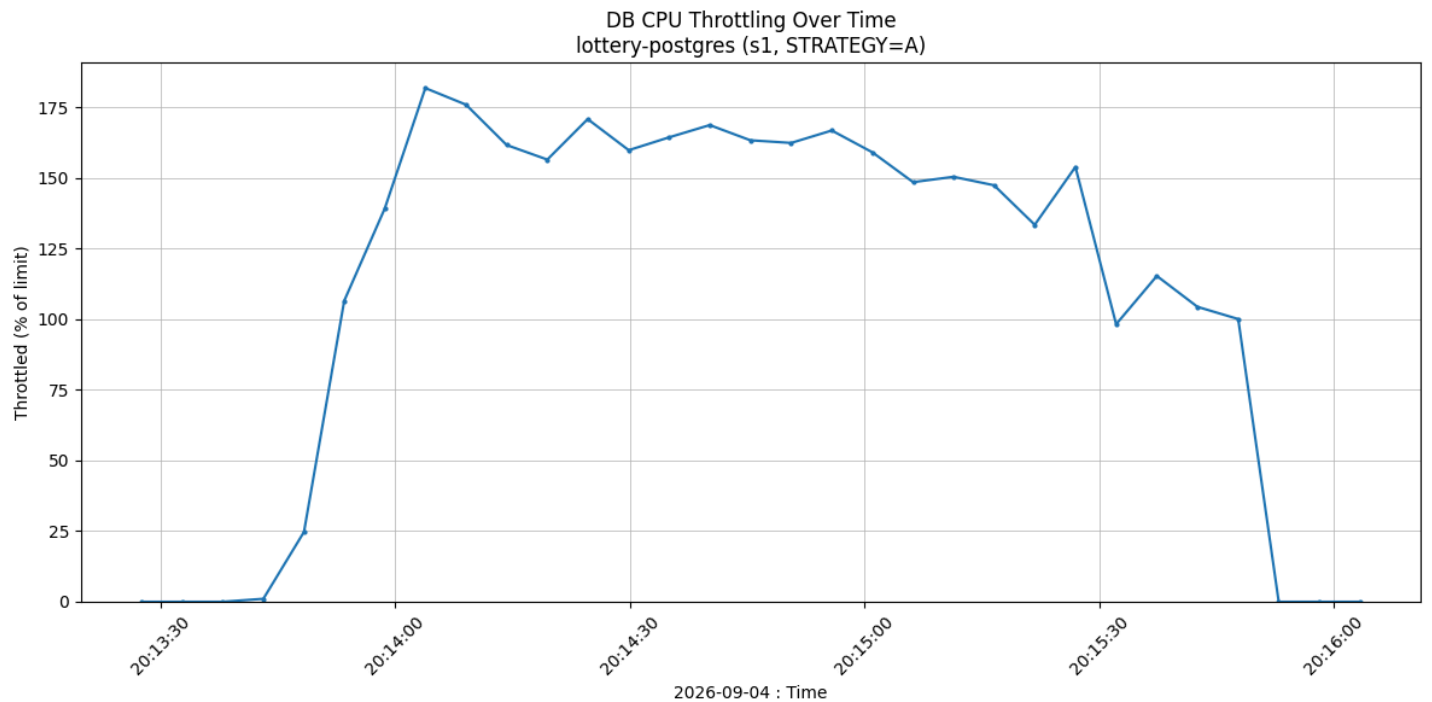


Strategy C — API pod ขนเพดาน เส้นนิ่งที่ 100 % ของ 500m ตลอดช่วงยิงโหลด เส้นเขียวคือค่า request ที่ตั้งไว้ (250m) นี้คือรูปของระบบที่ "ขยายต่อได้"

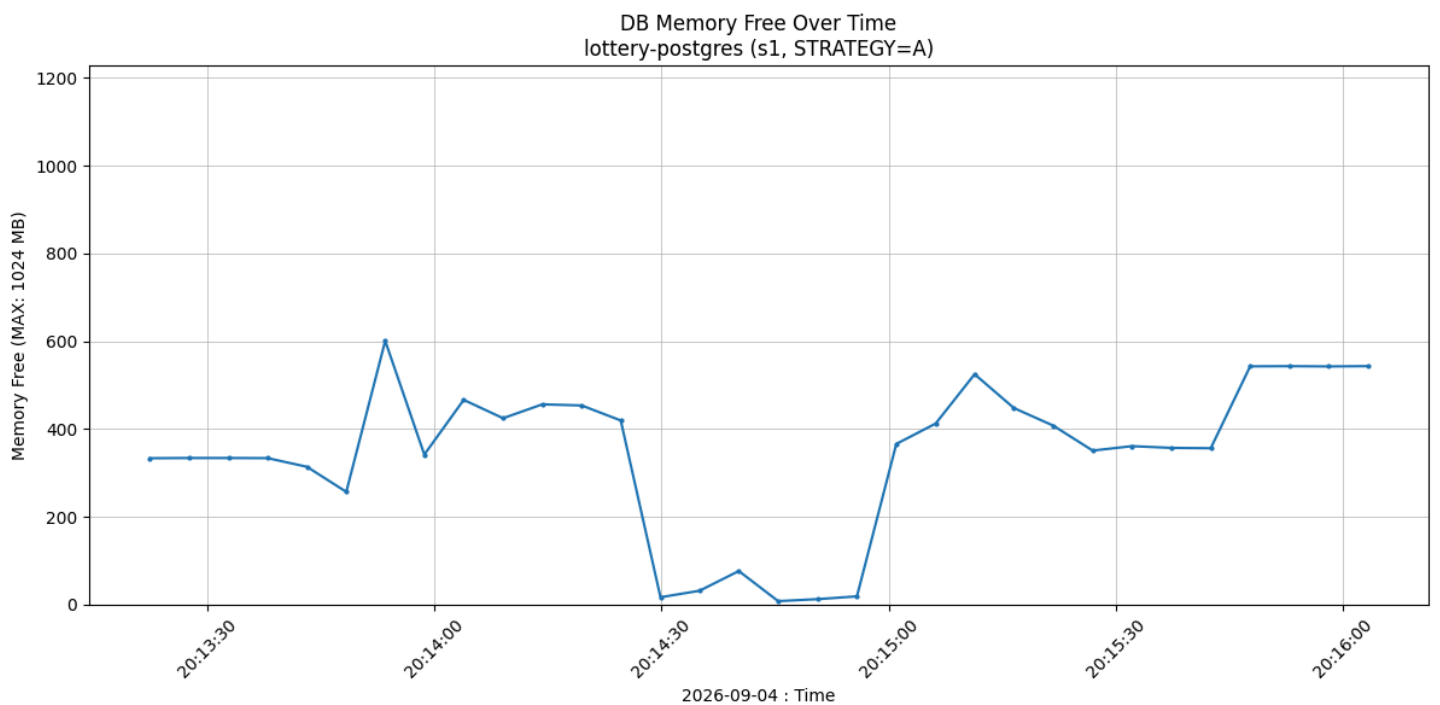


Strategy A เจ็บกว่าที่คิด

A ไม่ได้แค่ช้า แต่ทำให้ Postgres โดน *throttle* คือถูก cgroup ตัดโควตา CPU ทั้ง กราฟซ้ายอ่านว่าในหนึ่งวินาที cgroup ถูกปฏิเสธเวลา CPU ไปมากถึง 182 % ของ limit ที่มี นั่นคือกลไกที่ทำให้ p95 พุ่งไป 5,972 ms ส่วนกราฟขวาคือ memory ที่เหลือ ลงไปแตะ 8 MiB จาก 1 GiB



โดนตัดโควตา CPU สูงสุด 182 % ยิ่งตัวเลขสูง แปลว่า process ยิ่งถูกบังคับให้หยุดรอบ่อย



เหลือ 8 MiB จาก 1 GiB index 397 MB ของ Strategy A ใหญ่เกินกว่าที่ pod ขนาดนี้จะรับไหว

log จริงของ Strategy C

นี่คือส่วนสรุปจาก docs/evidence/s1-C/10-k6.log เครื่องหมาย ✓ หน้าเมตริกแปลว่า threshold ที่ตั้งไว้ผ่านทั้งหมด ในที่นี้คือ p95 ต่ำกว่า 100 ms และ p99 ต่ำกว่า 250 ms

● DOCS/EVIDENCE/S1-C/10-K6.LOG

✓ status 200

checks.....: 100.00% ✓ 641580 ✗ 0

data_received.....: 483 MB 4.0 MB/s

data_sent.....: 105 MB 877 kB/s

http_req_blocked.....: avg=803ns min=0s med=1µs max=567µs

p(90)=1µs p(95)=2µs

http_req_connecting.....: avg=21ns min=0s med=0s max=233µs

p(90)=0s p(95)=0s

✓ http_req_duration.....: avg=9.97ms min=320µs med=5.63ms max=142.1ms

p(90)=25.84ms p(95)=55.84ms

✓ { class:w0 }.....: avg=10.36ms min=320µs med=5.94ms max=138.55ms

p(90)=33.19ms p(95)=56.29ms

✓ { class:w3 }.....: avg=10.09ms min=349µs med=5.69ms max=142.1ms

p(90)=30.06ms p(95)=55.94ms

✓ { class:w4 }.....: avg=9.63ms min=358µs med=5.39ms max=92.35ms

p(90)=17.79ms p(95)=55.52ms

✓ { class:w5 }.....: avg=9.79ms min=350µs med=5.53ms max=93.18ms

p(90)=21.32ms p(95)=55.58ms

{ expected_response:true }...: avg=9.97ms min=320µs med=5.63ms max=142.1ms

p(90)=25.84ms p(95)=55.84ms

✓ http_req_failed.....: 0.00% ✓ 0 ✗ 641580

http_req_receiving.....: avg=10.5µs min=3µs med=8µs max=1.11ms

p(90)=18µs p(95)=28µs

http_req_sending.....: avg=3.35µs min=1µs med=2µs max=932µs

p(90)=6µs p(95)=8µs

http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s

p(90)=0s p(95)=0s

http_req_waiting.....: avg=9.95ms min=315µs med=5.62ms max=142.07ms

p(90)=25.83ms p(95)=55.83ms

http_reqs.....: 641580 5346.059128/s

iteration_duration.....: avg=10ms min=339.12µs med=5.67ms max=142.15ms

p(90)=25.88ms p(95)=55.88ms

iterations.....: 641580 5346.059128/s

running (2m00.0s), 000/100 VUs, 641580 complete and 0 interrupted iterations

log จริงของ baseline ที่ล้ม

รอบ baseline ไม่ได้แค่ช้าลง แต่ล้ม pool ขนาด 10 connection เต็มไปด้วย query ที่กินเวลาระดับวินาที ทำให้ /readyz ตอบไม่ทัน Kubernetes จึงถอด pod ออกจาก Service แล้ว request ที่เหลือก็ถูกตัดกลางคัน สังเกตว่า checks ผ่านแค่ 4.28 % และ 19,127 จาก 19,134 ครั้งที่พังเป็น error แบบ EOF

DOCS/EVIDENCE/S1-0/10-K6.LOG

```
checks.....: 4.28% ✓ 856          ✗ 19134
data_received.....: 641 kB 4.6 kB/s
data_sent.....: 3.2 MB 23 kB/s
http_req_blocked.....: avg=168.3µs min=0s      med=113µs max=6.62ms
p(90)=285µs p(95)=461µs
http_req_connecting.....: avg=147.82µs min=0s      med=97µs max=6.58ms
p(90)=252µs p(95)=420µs
✗ http_req_duration.....: avg=360.32ms min=0s      med=400µs max=22.55s
p(90)=1s p(95)=1.02s
✗ { class:w0 }.....: avg=542.82ms min=0s      med=404µs max=22.55s
p(90)=1.01s p(95)=1.05s
✗ { class:w3 }.....: avg=290.01ms min=0s      med=398µs max=15.24s
p(90)=7.2ms p(95)=1.02s
✗ { class:w4 }.....: avg=335.48ms min=0s      med=399µs max=16.5s
p(90)=1s p(95)=1.02s
✗ { class:w5 }.....: avg=274.3ms min=0s      med=397µs max=16.5s
p(90)=5.61ms p(95)=1.02s
{ expected_response:true }...: avg=6.74s min=478µs med=5.94s max=22.55s
p(90)=13.93s p(95)=16.34s
✗ http_req_failed.....: 95.71% ✓ 19134          ✗ 856
http_req_receiving.....: avg=2.81µs min=0s      med=0s max=3.28ms
p(90)=0s p(95)=0s
http_req_sending.....: avg=18.97µs min=0s      med=10µs max=5.38ms
p(90)=26µs p(95)=41µs
http_req_tls_handshaking.....: avg=0s min=0s      med=0s max=0s
p(90)=0s p(95)=0s
http_req_waiting.....: avg=360.3ms min=0s      med=380µs max=22.55s
p(90)=1s p(95)=1.02s
http_reqs.....: 19990 144.768259/s
iteration_duration.....: avg=360.57ms min=244.37µs med=679.5µs max=22.55s
p(90)=1s p(95)=1.02s
iterations.....: 19990 144.768259/s

running (2m18.1s), 000/100 VUs, 19990 complete and 0 interrupted iterations
```

บทเรียนที่เอาไปใช้ได้ทันที

เพราะฉะนั้นเราจึงแยก `/healthz` ที่เช็คแค่ว่า process ยังอยู่ ออกจาก `/readyz` ที่ ping database **liveness probe** ห้ามแตะ **database** ไม่งั้นตอน database แนน Kubernetes จะฆ่า pod ทั้งซ้ำเติม ส่วน readiness ควรแตะ เพื่อให้ pod ที่รับงานไม่ไหวถูกถอดออกจาก load balancer ชั่วคราว

05 S2 - CONTENTION

200 คนแย่ง pattern เดียวกัน 60 วินาที

นี่คือข้อที่โจทย์ให้ความสำคัญที่สุด ถ้าผู้ใช้ 200 คนค้น * * * * 2 3 พร้อมกัน ต้องไม่มีใครได้ตั๋วใบเดียวกัน กลไกที่ใช้คือคำสั่งเดียวจบใน transaction เดียว

```
WITH picked AS (  
  SELECT id FROM tickets  
  WHERE status = 0 AND <pattern>  
  LIMIT $2  
  FOR UPDATE SKIP LOCKED      -- ← หัวใจ: ใครล็อกแถวไหนไว้ คนถัดไปข้ามไปหยิบใบอื่น  
) , upd AS (  
  UPDATE tickets SET status = 1, reserved_by = $1 FROM picked WHERE tickets.id = picked.id  
  RETURNING id, number  
) , ledger AS (  
  INSERT INTO allocations (ticket_id, user_id) SELECT id, $1 FROM upd  
)  
SELECT id, number FROM upd;
```

`SKIP LOCKED` ทำให้ transaction ที่มาทีหลังข้ามแถวที่ถูกล็อกอยู่ แทนที่จะต่อคิวรอ ผลคือไม่มี lock contention latency จึงไม่ฟุ้ง และไม่มีโอกาสเกิด deadlock เพราะไม่มีใครรอใคร

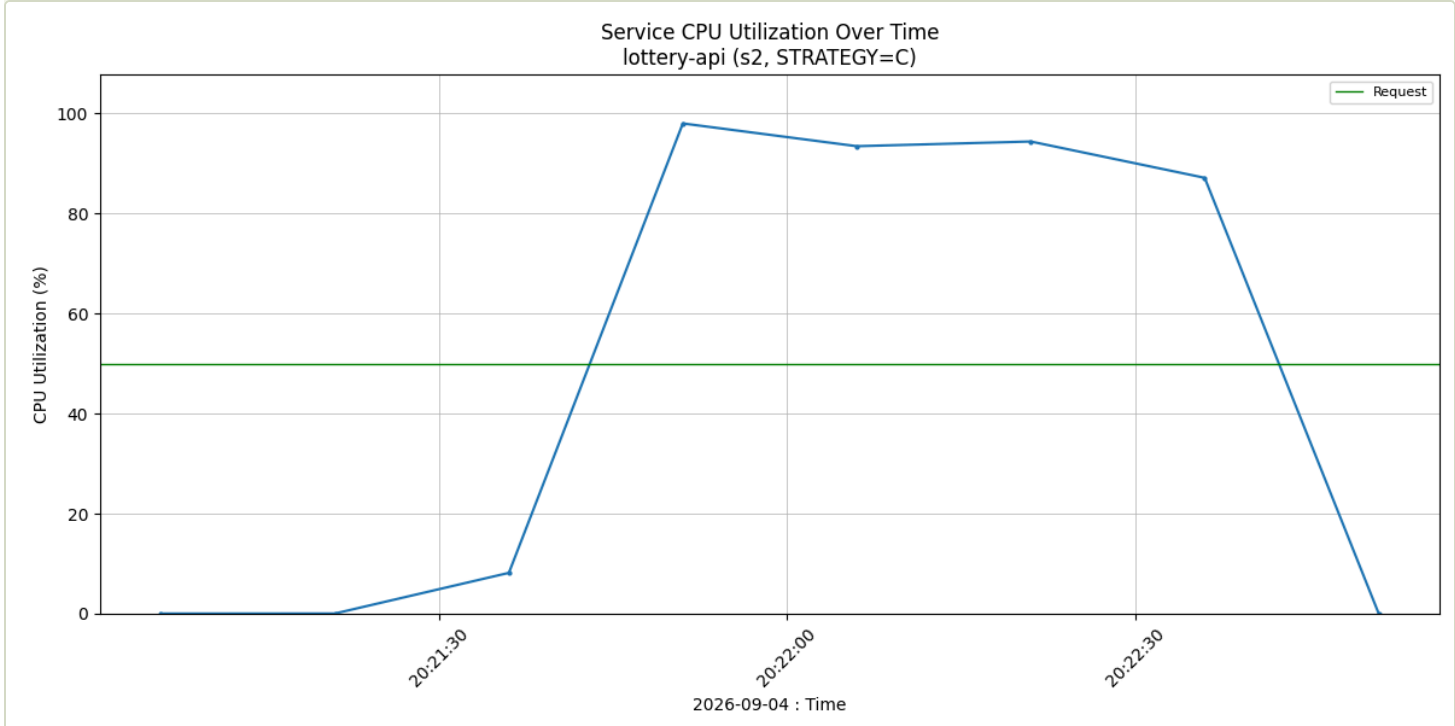
ตัวชี้วัด	PATTERN ****23 ของมีราว 100K ใบ	PATTERN **3*** ของมีราว 1M ใบ
จองได้ใน 60 วินาที	95,804 1,536/s	255,677 4,258/s
HTTP error	0	0
แถวใน ledger = ตัวที่ถูกจอง = จำนวน id ที่ไม่ซ้ำ	95,804	255,677
ตัวที่ถูกจ่ายซ้ำ	0	0
ledger ไม่ตรงกับสถานะตัว	0	0

ตัวชี้วัด	PATTERN *****23 ของมีราว 100K ใบ	PATTERN **3*** ของมีราว 1M ใบ
latency p95 / สูงสุด	548 ms / 2.5 s	74 ms / 467 ms
CPU API / Postgres	98 % / 101 %	98 % / 86 %

H2 ผ่านทั้งสองรอบ รวม 351,481 ใบ ไม่มีใบไหนไปถึงสองคน ตรวจสอบด้วย SQL บรรทัดเดียวที่ต้องได้ 0 แถว

```
● DOCS/EVIDENCE/S2B/30-VERIFY.TXT

ledger rows      : 255677
reserved tickets : 255677
distinct ticket  : 255677
DUPLICATES (H2, must be 0): 0
ledger vs status mismatch : 0
```



ตอนแย่งจองกัน คอขวดยังอยู่ที่ API ไม่ใช่ database API pod วิ่งชนเพดาน 98 % ขณะที่ Postgres อยู่ที่ 86 % แปลว่า SKIP LOCKED ไม่ได้สร้างการรอคิวที่ database ซึ่งเป็นผลลัพธ์ที่ต้องการ ถ้าใช้ FOR UPDATE เฉยๆ กราฟนี้จะกลับด้านทันที

ทำไมสองคอลัมน์ latency ต่างกันมาก

ไม่ใช่เพราะ concurrency พัง แต่เพราะ * * * * 2 3 ถูกจองไป 96 % ภายใน 60 วินาที ช่วงท้ายของรอบทดสอบ การสุ่มเลขจึงไปเจอแต่เลขที่ถูกจองไปแล้ว ต้องตกไปใช้วิธีสแกน pattern บน index ที่เต็มไปด้วยแถวตายแล้ว ส่วน * * 3 * * * มีของเป็นล้านใบ จองยังงี้ก็ไม่หมดใน 60 วินาที ตัวเลข 74 ms จึงเป็นภาพของสถานะปกติ ส่วน 548 ms คือภาพตอนของ ใกล้เคียง ซึ่งในระบบจริงควรจัดการที่ระดับ product เช่นขึ้นข้อความว่าเลขชุดนี้ ใกล้เคียงแล้ว

กับดักที่ เจอตอนทำ

ตอนแรกใช้ ORDER BY id LIMIT n FOR UPDATE SKIP LOCKED ตามสเปก ปรากฏว่าเกิดอาการ "หวัร่อน" คือแถวที่ถูกจองไปแล้วยังกองอยู่หน้าสุดของลำดับ id ทุกครั้งที่จองใหม่จึงต้องเดินผ่านของเก่าทั้งหมด วัดได้ว่าหลังจองไปแค่ 713 ใบ ต้องเดินผ่าน 68,453 แถว ใช้เวลา 145 ms ต่อครั้ง และแย่งเรื่อยๆ ทางแก้คือสุ่มเลขจริงที่อยู่ใน pattern เช่น * * * * 2 3 → 481723 แล้วยิง point lookup ถ้าพลาดค่อยลองใหม่สูงสุด 3 ครั้ง ต้นทุนจึงคงที่ไม่ว่าจะจองไปแล้วก็ใบ โดย SKIP LOCKED ยังเป็นตัวรับประกันความถูกต้องเหมือนเดิม

50 VUs ต่อเนื่อง 10 นาที ทั้ง search จอง และคืน

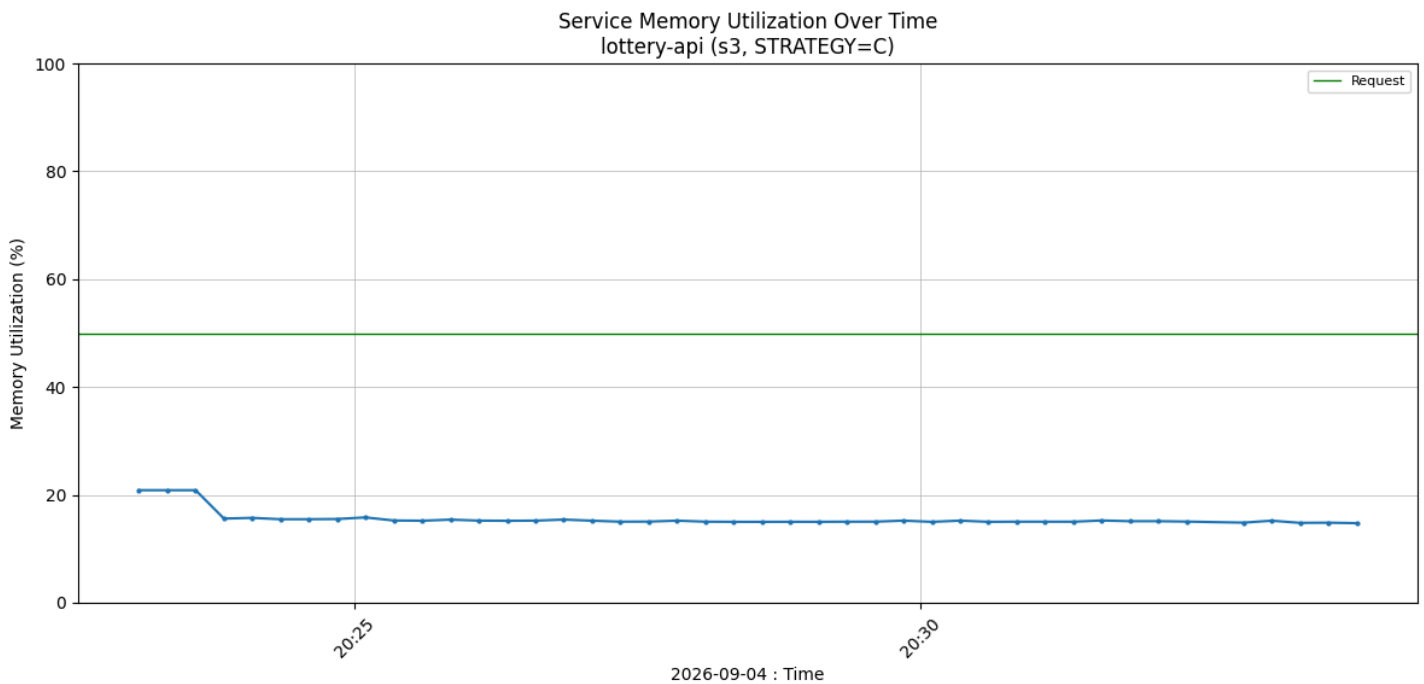
เป้าหมายของรอบนี้ไม่ใช่ความเร็ว แต่คือดูว่ารันยาวแล้ว memory รั่วหรือโดน OOMKilled ไหม โดยเฉพาะเมื่อ API pod ถูกจำกัดไว้แค่ 128Mi

ตัวชี้วัด	ผล	อ่านว่าอย่างไร
จำนวน request	743,994	1,240 req/s ตลอด 10 นาที
HTTP error	0	ไม่มี request ไหนพังเลย
latency p95 / สูงสุด	148 ms / 6.8 s	ไม่ผ่านเป้า 100 ms เพราะรอบนี้เขียนหนัก
memory ของ API pod สูงสุด	27 MiB	จาก limit 128 MiB เหลือที่ว่างอีกเยอะ
CPU ของ API pod สูงสุด	52 %	จาก 500m
restart / OOMKilled	0 / ไม่มี	ผ่านเกณฑ์ความเสถียร
แถวใน ledger	1,229,667	จากตัวที่ไม่ซ้ำกัน 1,089,126 ใบ
ถูกจองซ้ำหลังคืนของ	140,541	เป็นพฤติกรรมที่ถูกต้อง ไม่ใช่บั๊ก

แถวที่สถานะไม่สอดคล้อง / ledger ลอย

0 / 0

ข้อมูลไม่เพี้ยนเลยตลอด 10 นาที



memory ของ API pod ตลอด 10 นาที เส้นแบนอยู่ราว 21 % ของ limit ไม่ได้ขึ้นเรื่อยๆ ถ้ามี memory leak เส้นนี้จะค่อยๆ ไต่ขึ้นจนชนเพดานแล้วโดน OOMKilled คำนี้ได้มาจากการตั้ง `GOMEMLIMIT=100MiB` และ `GOMAXPROCS=1` ให้ Go runtime รู้ว่ามี cgroup limit อยู่

ตัวเลขที่เกือบอ่านผิดเป็นบั๊ก

ตอนแรกสคริปต์ตรวจรายงานว่ามีตัวซ้ำ 129,739 ใบ ซึ่งดูเหมือน H2 พัง แต่ที่จริงเป็น **query** ตรวจที่เขียนผิดเอง เพราะ S3 เป็น scenario เดียวที่เรียก `/v1/release` ตัวที่ถูกคืนแล้วถูกจองใหม่เป็นสิ่งที่ถูกต้อง และ ledger แบบ append-only ก็บันทึกทั้งสองครั้งตามจริง ตอนนี้ S3 จึงใช้ชุดตรวจคนละแบบ คือดูความสอดคล้องของสถานะและ ledger ลอย ซึ่งได้ 0 ทั้งคู่ ส่วน H2 พิสูจน์ด้วย S2 ที่ไม่มีการคืนของเลย

✓ status 200

```

checks.....: 100.00% ✓ 743994      ✗ 0
data_received.....: 306 MB   509 kB/s
data_sent.....: 133 MB   222 kB/s
http_req_blocked.....: avg=1.82µs   min=0s      med=1µs     max=8.06ms
p(90)=2µs      p(95)=3µs
http_req_connecting.....: avg=53ns    min=0s      med=0s      max=999µs
p(90)=0s      p(95)=0s
✗ http_req_duration.....: avg=40.24ms min=1.12ms  med=16.53ms max=6.79s
p(90)=52.93ms p(95)=147.94ms
  { expected_response:true }...: avg=40.24ms min=1.12ms  med=16.53ms max=6.79s
p(90)=52.93ms p(95)=147.94ms
✓ http_req_failed.....: 0.00% ✓ 0      ✗ 743994
http_req_receiving.....: avg=20.95µs min=4µs     med=14µs    max=33.89ms
p(90)=34µs    p(95)=48µs
http_req_sending.....: avg=7.77µs   min=1µs     med=5µs     max=8.69ms
p(90)=11µs    p(95)=15µs
http_req_tls_handshaking.....: avg=0s      min=0s      med=0s      max=0s
p(90)=0s      p(95)=0s
http_req_waiting.....: avg=40.21ms  min=1.11ms  med=16.5ms  max=6.79s
p(90)=52.9ms  p(95)=147.91ms
http_reqs.....: 743994   1239.986716/s
iteration_duration.....: avg=120.96ms min=20.79ms med=60.85ms max=6.87s
p(90)=257.97ms p(95)=347.61ms
iterations.....: 247998   413.328905/s

```

running (10m00.0s), 00/50 VUs, 247998 complete and 0 interrupted iterations

07 บทเรียนสำหรับ JUNIOR

สิ่งที่ผิดพลาดระหว่างทาง มีค่ากว่าตัวเลขที่สวยงาม

ระหว่างทำ benchmark ชุดนี้มีความผิดพลาดสามอย่างที่ทุกอย่างดูเหมือนจะทำงานได้ปกติ ถ้าไม่ตรวจให้ดีจะได้ตัวเลขที่ผิดไปเลย

สิ่งที่เกิดขึ้น	อาการที่เห็น	วิธีป้องกัน
ยิงผิดเป้า k6 ไปที่ port ของ dev server ที่ปิดอยู่	ได้ 4 ล้าน request ที่ 68,000/s พร้อม summary หน้าตาปกติ แต่ที่จริงคือ connection ถูกปฏิเสธทั้งหมด	ใส่ preflight เช็คค่า <code>/healthz</code> ตอบ 200 ก่อนเริ่มยิงทุกครั้ง

สิ่งที่เกิดขึ้น	อาการที่เห็น	วิธีป้องกัน
เวลาเป็นจำนวนเต็ม date +%s คินค่าเป็นวินาทีเต็ม	ช่วงเวลาจริง 5.2 วินาทีถูกหารด้วย 5 ทำให้ทุกกราฟที่คิดจากอัตราสูงเกินจริง 4 % เห็นเป็น CPU 104 % ทั้งที่ limit บังคับไว้ 100 %	ใช้เวลาที่มีทศนิยม และส่งสั้ยไว่ก่อนเมื่อเห็นค่าที่เกินเพดานที่เป็นไปไม่ได้
ตรวจผิดเงื่อนไข ใช้ query เดียวกับทุก scenario	soak test รายงานตัวซ้ำ 129,739 ใบ เหมือนระบบพัง ทั้งที่เป็นการจองใหม่หลังคืนของตามปกติ	เขียน invariant ให้ตรงกับสิ่งที่ scenario นั้นทำจริง อย่าใช้ชุดตรวจเดียวกับทุกกรณี

สามข้อที่เอาไปใช้กับงานอื่นได้เลย

- ต้องมีตัวเปรียบเทียบเสมอ ถ้าไม่ได้วัด baseline ที่ไม่มี index เลย ก็ไม่มีทางรู้ว่า 56 ms ที่ได้นั้นดีจริงหรือแค่ดูดี
- วัดว่าอะไรเดิมก่อน ไม่ใช่แค่วัดว่าเร็วแค่ไหน latency บอกว่าตอนนี้เป็นอย่างไง แต่กราฟ CPU บอกว่าขยายต่อทางไหนได้
- ผลที่ไม่ผ่านคือข้อมูล ไม่ใช่ความล้มเหลว การที่ Strategy A ทำให้ Postgres เหลือ memory 8 MiB คือหลักฐานชั้นที่ดีที่สุดว่าทำไมถึงไม่ควรเลือกมัน

08 คำศัพท์

ศัพท์ที่โผล่ในรายงานนี้

p95	เวลาที่ 95 % ของ request เร็วกว่าค่านี้ ใช้แทนค่าเฉลี่ยเพราะค่าเฉลี่ยจะซ่อน request ช้าๆ ส่วนน้อยที่ผู้ใช้รู้สึกได้จริง
VU	virtual user คือผู้ใช้จำลองหนึ่งคนใน k6 ที่ยิง request วนไปเรื่อยๆ 200 VUs คือจำลอง 200 คนพร้อมกัน
SKIP LOCKED	ตัวเลือกของ SELECT ... FOR UPDATE ที่สั่งว่าถ้าแถวไหนถูก transaction อื่นล็อกอยู่ ให้ข้ามไปเลยแทนที่จะรอ เหมาะกับการแจกของจากคิว
BitmapAnd	วิธีที่ Postgres รวมผลจากหลาย index เข้าด้วยกัน จะคุ้มก็ต่อเมื่อแต่ละ index คัดของทั้งได้เยอะ ซึ่ง index ของเลขหลักเดียวคัดได้แค่ 1 ใน 10 จึงมักไม่คุ้ม

working set	memory ที่ใช้จริง โดยไม่นับ page cache ที่ระบบเรียกคืนได้ ถ้าดูแค่ตัวเลข memory ดิบ Postgres จะดูเหมือนใช้เกือบเต็ม limit ตลอดเวลา เพราะมันเอา memory วางไปทำ cache
CPU throttling	เมื่อ container ใช้ CPU เกินโควตาที่ตั้งไว้ kernel จะบังคับให้หยุดรอจนหมดรอบเวลา ยิ่งโดนบ่อย latency ยิ่งพุ่ง
liveness / readiness	liveness ตอบว่า process ยังมีชีวิตอยู่ไหม ถ้าตอบไม่ผ่าน Kubernetes จะฆ่าทิ้งแล้วสร้างใหม่ ส่วน readiness ตอบว่าตอนนี้พร้อมรับงานไหม ถ้าไม่ผ่านจะแค่ถูกถอดออกจาก load balancer ชั่วคราว
exit code 99	ค่าที่ k6 คืนเมื่อรันจบครบแต่มี threshold ที่ตั้งไว้ไม่ผ่าน ไม่ได้แปลว่าการทดสอบพัง

09 ข้อจำกัด

สิ่งที่ตัวเลขชุดนี้ยังไม่ได้พิสูจน์

- ทุกอย่างรันบน docker-desktop node เดียว ทั้ง API, Postgres และ k6 จึงแย่ง CPU กันเอง ตัวเลข throughput สัมบูรณ์จะต่างออกไปถ้าแยกเครื่อง แต่การเปรียบเทียบระหว่าง **strategy** ซึ่งเป็นแกนของข้อสรุป ไม่ได้รับผลกระทบ
- กราฟ disk free วัดทั้ง volume ของ VM ไม่ใช่ device เฉพาะของ database เพราะ PVC ถูก back ด้วย host path จึงใช้ดูคร่าวๆ เท่านั้น
- กราฟ CPU และ memory ของ API มาจาก metrics-server ที่อัปเดตทุกราว 15 วินาที ส่วนของ Postgres อ่านจาก cgroup ทุก 5 วินาที ความละเอียดจึงไม่เท่ากัน โดยตั้งใจ
- ยังไม่ได้ทดสอบกรณีที่ของถูกจองไปแล้ว 80 % ตั้งแต่ต้น ซึ่งจะกระทบ selectivity ของ partial index
- PoC นี้ไม่มี authentication, payment, ระบบงวด, replication และ caching layer ตามที่ระบุไว้ว่าอยู่นอกขอบเขต

ตัวเลขทุกตัวในรายงานนี้มาจากไฟล์ใน `docs/evidence/` แต่ละ scenario มี 19 ไฟล์ ประกอบด้วย log ของ k6 แบบเต็ม, summary เป็น JSON และ HTML, กราฟ 9 รูป, ข้อมูลดิบ CSV ที่ใช้วาดกราฟ และผลตรวจ SQL รันซ้ำได้ด้วย `make capture-all` และวาดกราฟใหม่จากข้อมูลเดิมได้ด้วย `make charts`
DIR=docs/evidence/s1-C

